

Dakshin Delights

Fast Restaurant Assistant

Developer & User Manual

“A warm, upbeat AI host that greets guests, shows the menu, takes orders, handles dietary preferences, and confirms bills.”

Contents

1	Introduction	3
1.1	Key Capabilities	3
1.2	Architecture at a Glance	3
2	Installation & Setup	3
2.1	Prerequisites	3
2.2	Install Dependencies	3
2.3	Environment File	4
2.4	Run the Assistant	4
3	User Manual — How to Chat	4
3.1	Special Meta-Commands (type exactly)	4
3.2	Viewing the Menu	4
3.3	Adding Items	4
3.4	Removing Items	4
3.5	Dietary Preferences	5
3.6	Recommendations	5
3.7	Billing & Checkout	5
3.8	FAQ	5
4	Code Reference — Section by Section	5
4.1	Section 1 — Menu Data & Categories	5
4.1.1	MENU_DETAILS	5
4.1.2	Category Lists (IDLI_ITEMS, DOSA_ITEMS, ...)	6
4.1.3	CATEGORY_SYNONYMS	6
4.1.4	POPULAR_ITEMS, DESCRIPTIONS	6
4.2	Section 2 — Availability & Specials	6
4.2.1	load_availability_and_specials()	6
4.3	Section 3 — Lookup & Category Helpers	6
4.3.1	_norm(s) — Text Normalisation	6
4.3.2	_base(key) — Base Item Name	7
4.3.3	match_candidates(query)	7
4.3.4	fuzzy_best	7
4.3.5	parse_preference(text)	7
4.3.6	item_matches(item, parsed)	8
4.3.7	show_menu_text / render_full_menu / render_category	8
4.3.8	menu_search	8
4.4	Section 4 — Recommendations	8
4.4.1	build_recommendations(budget, group_size, preference, cart, saved_pref)	8
4.4.2	todays_specials_text(s)	8
4.4.3	dietary_listing(text, s)	8
4.4.4	FAQ	9
4.5	Section 5 — Session State	9
4.6	Section 6 — Tool Schemas and Handlers	9
4.6.1	LangChain @tool Decorators	9
4.6.2	handle_tool(name, args, s)	10
4.6.3	show_bill logic	10

4.6.4	confirm_order logic	10
4.6.5	view_analytics	10
4.7	Section 7 — The Agent (LLM Integration)	10
4.7.1	Model Initialisation	10
4.7.2	system_prompt(s)	10
4.7.3	run_turn(s, user_text)	10
4.7.4	extract_text(msg)	11
4.8	Section 8 — Deterministic Command Parser	11
4.8.1	Why a Deterministic Parser?	11
4.8.2	interpret_command(text, s)	11
4.8.3	parse_items(text)	11
4.8.4	resolve_pending(text, s)	12
4.8.5	handle_correction(t, s)	12
4.8.6	order_followup(s)	12
4.8.7	Utility Detectors	12
4.9	Streaming Output	12
5	Data Files	12
5.1	item_availability.json	12
5.2	analytics_dashboard.json	13
6	Extending the Bot	13
6.1	Adding a New Menu Item	13
6.2	Adding a New Dietary Tag	13
6.3	Adding a New FAQ Entry	13
6.4	Changing the LLM	14
7	Troubleshooting	14
8	Glossary	14

Introduction

Dakshin Delights Assistant is a command-line chatbot that acts as a restaurant host for a South-Indian eatery. It combines a *deterministic fast-path parser* (for unambiguous commands like “add 2 masala dosa”) with a **Gemini LLM** (via LangChain) for natural conversation, recommendations, and small-talk.

Key Capabilities

- Display the full menu or individual categories (Idli, Dosa, Rice, etc.)
- Fuzzy-match item names tolerant of typos and alternate spellings
- Track a live in-memory shopping cart per session
- Detect & store dietary preferences (vegan, Jain, gluten-free, mild ...)
- Generate budget combos, group bundles, and today’s-specials
- Produce a formatted bill with 5% GST and confirm/cancel orders
- Log confirmed orders to a JSON analytics file
- Answer common restaurant FAQs (hours, location, payment, Wi-Fi ...)

Architecture at a Glance

Layer	What it does
Menu data	Python dicts/lists — prices, tags, categories
Availability layer	JSON file read on every call for live stock
Helper utilities	Fuzzy matching, preference parsing, category resolution
Tool handlers	Pure-Python functions wired to LangChain <code>@tool</code> schemas
Deterministic parser	Regex + keyword logic — handles adds/removes/bill without LLM
LLM layer	Gemini via <code>ChatGoogleGenerativeAI</code> for natural conversation
Session	Single dataclass holding cart, preference, history, pending state
Main loop	<code>input()</code> REPL with streaming word-by-word output

Installation & Setup

Prerequisites

- Python 3.10 or higher
- A Google Gemini API key
- (Optional) `rapidfuzz` for faster fuzzy matching

Install Dependencies

```
pip install langchain langchain-google-genai python-dotenv rapidfuzz
```

Environment File

Create a `.env` file in the project root:

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

Run the Assistant

```
python dakshin_delights.py
```

User Manual — How to Chat

Special Meta-Commands (type exactly)

Command	Effect
menu	Instantly prints the full menu (bypasses LLM)
cart	Shows current cart contents
history	Shows the action log for this session
preferences	Shows the saved dietary preference
exit / quit	Ends the session

Viewing the Menu

- **Full menu:** “menu”, “show me everything”, “what do you have”
- **Category:** “show dosa options”, “dosa menu”, “desserts”, “what drinks do you have?”

Adding Items

Any of the following patterns work:

```
add 2 masala idli
give me one podi masala dosa
i'll have ghee plain dosa and badam milk
order 3 gulab jamun and lemon rice
```

If your item name matches multiple dishes (e.g. “idli”), the bot will ask you to choose from a numbered list. Reply with the number or the full name.

Removing Items

```
remove masala idli
cancel 2 gulab jamun
take off lemon rice
```

Dietary Preferences

```
i'm vegan  
no onion please  
jain food only  
i don't like garlic  
make it mild
```

Once set, the bot filters all recommendations and menus through your preference for the rest of the session. Reset by stating a new preference.

Recommendations

```
what do you recommend?  
suggest something under 200 rupees  
combo for a group of 4  
what's good today?  
today's specials
```

Billing & Checkout

bill	-> Shows itemised bill with GST
checkout	
yes / confirm	-> Places the order (after bill)
no / cancel	-> Keeps editing (after bill)

FAQ

The bot answers common questions directly:

- Hours, location, delivery, payment methods
- Wi-Fi, parking, table reservations, GST info
- Spice-level adjustments

Code Reference — Section by Section

Section 1 — Menu Data & Categories

MENU_DETAILS

```
1 MENU_DETAILS = {  
2     "idli (2 pcs)": {"price": 45, "tags": ["vegan", "vegetarian",  
3                                     "jain", "gluten-free"]},  
4     ...  
5 }
```

What it is: The single source of truth for the entire menu. Each key is the canonical item name (lowercase), and the value holds the price (in INR) and a list of dietary tags.

Why it exists: Centralising the data here means that billing, searching, filtering, and display all read from one place; no values are hard-coded twice.

Category Lists (IDLI_ITEMS, DOSA_ITEMS, ...)

Each list groups the canonical item names into a logical section of the menu. They are used for:

- Rendering sections in the full menu printout (MENU_GROUPS)
- Category-based search (CATEGORY_ITEMS dict)
- Finding “similar” in-stock alternatives when an item is sold out

CATEGORY_SYNONYMS

Maps many natural-language words to a canonical category key:

```
1 "dosai": "dosa", "crepe": "dosa", "biryani": "rice", ...
```

Why: Users type things like “show me dosai options” or “any biriyani?”. Without this map those queries would silently return nothing.

POPULAR_ITEMS, DESCRIPTIONS

Curated list of best-sellers and short appetising blurbs used by the recommendation engine to rank items and format the output attractively.

Section 2 — Availability & Specials

load_availability_and_specials()

```
1 def load_availability_and_specials() -> dict:
2     ...
```

Reads `item_availability.json` on *every call*. The file stores:

- `availability` — a dict mapping every item name to `true/false`
- `todays_specials` — a list of highlighted items

Why re-read every call? So a manager can edit the JSON while the bot is running (mark items sold-out, change specials) and the change takes effect on the very next customer message — no restart required.

Fallback behaviour: If the file is missing or malformed, all items default to available and a one-time warning is printed (the `_AVAIL_WARNED` flag prevents log-spam on every subsequent call).

Section 3 — Lookup & Category Helpers

_norm(s) — Text Normalisation

```
1 def _norm(s: str) -> str:
2     s = s.replace("idly", "idli").replace("pudi", "podu")
3     s = re.sub(r"[\(\)]", " ", s)
4     ...
```

Converts any string into a canonical form for comparison:

- Lower-case everything
- Unify alternate spellings: idly→idli, dosai→dosa, pudi→podi
- Expand piece-count tokens: "4pc" → "4 pcs"
- Strip parentheses (but keep content) and collapse whitespace

`_base(key)` — Base Item Name

Strips piece-count tokens to get the item's base form. Used when checking whether two variants refer to the same dish (e.g. *idli (2 pcs)* and *idli (4 pcs)* both have base "idli").

`match_candidates(query)`

```
1 def match_candidates(query: str) -> List[str]:
2     ...
```

Returns *all* menu keys a user query could mean, enabling disambiguation.

Matching cascade (in priority order):

1. Exact match against raw key
2. Exact match against normalised key
3. All tokens in query are a subset of the item's tokens ("idli 4" → "idli (4 pcs)")
4. Substring of normalised key
5. Fuzzy match via `rapidfuzz` (score 72)

Returns [] = not found, [x] = unambiguous, [x,y,...] = ambiguous.

`fuzzy_best`

Uses `rapidfuzz.process.extractOne` if available, otherwise falls back to Python's built-in `difflib.get_close_matches`. Lets users type "masla dosa" or "gulab jamn" and still get the right item.

`parse_preference(text)`

```
1 def parse_preference(text: str) -> dict:
2     ...
```

Parses free text into a structured filter with four fields:

Field	Meaning
<code>require</code>	Tags the item MUST have (e.g. "vegan")
<code>exclude_tags</code>	Tags the item must NOT have (e.g. "spicy")
<code>exclude_words</code>	Words that must not appear in the item name
<code>notes</code>	Human-readable labels for acknowledgement messages

Why Jain == no onion + no garlic: The menu has no per-ingredient database. The "jain" tag on items is the only guarantee of onion-and-garlic-free preparation. Any item tagged "jain" is also name-checked to exclude items like *Ghee Onion Dosa*.

`item_matches(item, parsed)`

Boolean check of all three filter fields against a single menu item. Used throughout the recommendation and dietary-listing code.

`show_menu_text / render_full_menu / render_category`

Rendering helpers that iterate the menu group lists, call `load_availability_and_specials()` for live status, and produce formatted Unicode strings (with sold-out markers, section headers).

`menu_search`

A general-purpose search function accepting free text, a max price, and/or a dietary preference tag. Passes all three filters simultaneously.

Section 4 — Recommendations

`build_recommendations(budget, group_size, preference, cart, saved_pref)`

Three operating modes:

1. **Budget combo** — Greedily picks a main (dosa/rice), a side (drink/sweet), and optional fillers whose total price $\leq$ budget. Reports the amount saved.
2. **Group bundle** — Scales servings of one dosa, one idli, half the group in sweets, and one drink per person.
3. **Default** — Today's specials + one chef's pick per section (dosa, light/idli, dessert/-drink) sorted by popularity. Appends a pairing suggestion if the cart already has items.

All three modes respect the user's saved/parsed dietary preference via `item_matches` and show only currently available items.

`todays_specials_text(s)`

Dedicated function for "what's special today". Resolves special names to canonical keys, filters by preference and availability, and returns a preference-aware formatted list.

`dietary_listing(text, s)`

Parses a dietary filter from the user's text, then walks all menu groups to produce a sectioned listing of matching available items. Used for queries like "show me all vegan options" or "list gluten-free dishes".

FAQ

The FAQ list stores (keyword_list, answer) pairs. The `faq_answer(t)` function returns the first matching answer or None (to fall through to the LLM).

Section 5 — Session State

```

1 @dataclass
2 class Session:
3     cart: Dict[str, int] = field(default_factory=dict)
4     preference: str = ""
5     history: List[str] = field(default_factory=list)
6     awaiting_confirmation: bool = False
7     pending: Optional[dict] = None
8     messages: List[BaseMessage] = field(default_factory=list)

```

Field	Purpose
cart	Canonical item name → quantity mapping
preference	Saved dietary preference string (e.g. "vegan")
history	Human-readable log of add/remove actions
awaiting_confirmation	Flag set after <code>show_bill</code> ; gates YES/NO confirmation
pending	Stores a queued disambiguation (<i>choose</i>) or swap (<i>swap</i>) awaiting the user's reply
messages	Full LangChain message history for multi-turn LLM context

Why pending? When a query like “add idli” matches multiple variants, the bot stores the candidates here and waits for the user's choice rather than guessing. The next user message is routed through `resolve_pending()` before any other logic.

Section 6 — Tool Schemas and Handlers

LangChain @tool Decorators

Nine tool schemas are declared with `@tool`:

Tool	Purpose
<code>show_menu</code>	Full or category menu
<code>search_menu</code>	Filtered search (text + price + tag)
<code>recommend</code>	Budget combos / group bundles / default picks
<code>add_item</code>	Adds a qty of an item to the cart
<code>remove_item</code>	Removes a qty (or all) of an item
<code>set_preference</code>	Saves the dietary preference
<code>show_bill</code>	Renders itemised bill with GST
<code>confirm_order</code>	Persists order or cancels
<code>view_analytics</code>	Shows order-count, revenue, top item

The `TOOL_SCHEMAS` list is bound to the LLM via `llm.bind_tools(TOOL_SCHEMAS)`. When the model decides to call a tool, the `handle_tool()` dispatcher executes the corresponding Python logic and returns a string result.

handle_tool(name, args, s)

Central dispatcher. For each tool name it calls the appropriate helper with arguments extracted from `args` and the live `Session` object. Crucially, **tool outputs are shown verbatim** to the user — the model only supplies a warm lead-in sentence, never re-types prices or item lists.

show_bill logic

Computes subtotal, 5% GST, grand total (float), and a floor-rounded bill value (for easy cash payment), then sets `s.awaiting_confirmation = True`. The next user YES/NO routes through the confirmation gate in `interpret_command`.

confirm_order logic

On confirmation, saves a JSON record to `analytics_dashboard.json`, clears the cart, history, and confirmation flag. On cancellation, simply unsets the flag.

view_analytics

Reads all saved orders, sums revenue, counts per-item quantities, and reports total orders, revenue, average order value, and most popular item.

Section 7 — The Agent (LLM Integration)

Model Initialisation

```
1 llm = ChatGoogleGenerativeAI(model="gemini-3.5-flash", temperature=0)
2 llm_with_tools = llm.bind_tools(TOOL_SCHEMAS)
```

`temperature=0` gives deterministic, consistent responses. Tool binding makes the model aware of all nine tools and lets it emit structured `tool_call` blocks.

system_prompt(s)

Built fresh on every LLM call. Injects live state into the model's context:

- Current cart contents
- Saved preference
- Whether a bill is awaiting confirmation
- Today's specials
- All sold-out items

This approach avoids the model reasoning on stale data between turns.

run_turn(s, user_text)

The agentic loop:

1. Appends user message to `s.messages`

2. Invokes the model
3. If no tool calls → return the model’s text
4. If tool calls → capture a short lead-in from the model’s text, execute each tool via `handle_tool`, append `ToolMessage` results, and repeat (up to `max_steps=4`)
5. Assembles final reply: lead-in + tool outputs + cart summary (if changed)

`extract_text(msg)`

Strips Gemini-specific `tool_use` and metadata blocks from the raw `AIMessage.content`, which can be a mix of strings and dicts. Returns only human-readable text.

Section 8 — Deterministic Command Parser

Why a Deterministic Parser?

LLMs can misroute unambiguous commands (model drift, token-count pressure). The deterministic layer handles every clear order without calling the LLM at all, so “add 2 masala dosa” *always* works — regardless of model behaviour.

`interpret_command(text, s)`

Routes the user input through a priority-ordered decision tree:

1. **Pending** — resolve a queued disambiguation or swap
2. **Bill confirmation gate** — YES/NO when awaiting confirmation
3. **Bill trigger** — “bill”, “checkout”, “done ordering” ...
4. **Correction cues** — “I said X not Y”, “change to X”
5. **Specials** — “today’s special”, “any specials?”
6. **Dietary listing** — “show all jain dishes”
7. **Preference statement** — “I’m vegan”, “no garlic”
8. **Recommendations** — “recommend”, “budget under 150” ...
9. **Menu display** — “menu”, “show dosa options” ...
10. **Add/Remove** — parsed item list with quantities
11. **Bare item list** — no verb, but items found (upsell entry point)
12. **FAQ** — hours, location, delivery, payment ...
13. Returns `None` → LLM handles it (greetings, chit-chat)

`parse_items(text)`

Splits the user text on “and”, commas, “&”, “plus”; then for each chunk:

- Strips add/remove verbs and filler words
- Extracts a leading digit or number word as quantity

- Calls `match_candidates()` to classify as matched / unmatched / ambiguous
- Skips chunks that contain negation/dislike cues (e.g. “no onion”)

Returns three lists: matched (`key`, `qty`) pairs, unmatched raw strings, and ambiguous (`raw`, `candidates`, `qty`) tuples.

`resolve_pending(text, s)`

Choose flow: The user replies with a number (“2”), an ordinal (“first”), or a name fragment. The resolver picks the best candidate and calls `add_item`.

Swap flow: After a correction (“I said idli 4 pcs, not 2 pcs”), the bot confirms the swap before executing, to avoid unwanted changes.

`handle_correction(t, s)`

Detects correction cues (“I said”, “change to”, “instead of”) and attempts to find a sibling variant already in the cart (same base name, different variant). If found, queues a swap confirmation; otherwise adds the wanted item directly.

`order_followup(s)`

After a successful add, generates a gentle upsell nudge if the cart has no dessert or drink yet. Respects the user’s saved dietary preference.

Utility Detectors

- `detect_budget(t)` — regex for “under/below/within N”
- `detect_group(t)` — regex for “group of N / for N people”
- `has_quantity(t)` — checks for digits or number words
- `is_full_menu_request(t)` — matches “full”, “all”, “menu”, ...

Streaming Output

```
1 def stream_out(text: str, word_delay: float = 0.018):
2     for tok in re.findall(r"\S+|\s+", text):
3         sys.stdout.write(tok); sys.stdout.flush()
4         if tok.strip(): time.sleep(0.018)
```

Prints the reply word-by-word with an 18 ms pause between words, simulating the feel of a streaming AI response in the terminal. Whitespace tokens are printed instantly (no pause) to preserve formatting.

Data Files

`item_availability.json`

Auto-created on first run. Edit this file while the bot is running to:

- Mark items sold-out: "podi masala dosa": false
- Change today's specials list

Schema:

```
{
  "todays_specials": ["Ghee Onion Dosa (2 pcs)", ...],
  "availability": {
    "idli (2 pcs)": true,
    "podi masala dosa": false,
    ...
  }
}
```

analytics_dashboard.json

Appended on every confirmed order. Each entry:

```
{
  "order_id": 101,
  "timestamp": "2025-07-21 18:45:03",
  "items": {"podi masala dosa": 2, "badam milk": 1},
  "total": 375
}
```

Extending the Bot

Adding a New Menu Item

1. Add the item to MENU_DETAILS with price and tags.
2. Append the name to the appropriate category list (IDLI_ITEMS, DOSA_ITEMS, ...).
3. Optionally add a short blurb to DESCRIPTIONS.
4. The availability system picks it up automatically on next run.

Adding a New Dietary Tag

1. Tag the relevant items in MENU_DETAILS.
2. Add detection logic to parse_preference() (require or exclude).
3. Add a synonym to CATEGORY_SYNONYMS if needed.

Adding a New FAQ Entry

Append a tuple to the FAQ list:

```
1 (["keyword1", "keyword2"], "Your answer here.")
```

Changing the LLM

Swap the `ChatGoogleGenerativeAI` initialisation for any `LangChain BaseChatModel` that supports tool/function calling (e.g. `ChatOpenAI`, `ChatAnthropic`).

Troubleshooting

Symptom	Resolution
“An error occurred: ...” at startup	Check <code>GOOGLE_API_KEY</code> in <code>.env</code>
Bot can’t find an item	Check spelling; the fuzzy matcher needs at least 72% similarity
Sold-out item still appearing	Edit <code>item_availability.json</code> — set value to <code>false</code>
Bill shows wrong items	Type <code>cart</code> to inspect; use <code>remove</code> commands to correct
Analytics file empty	Only confirmed (YES after bill) orders are saved
Availability file warning on startup	Validate the JSON in <code>item_availability.json</code>

Glossary

Canonical name

The exact lowercase string used as a key in `MENU_DETAILS`.

Disambiguation

The process of asking the user to choose between multiple menu items that match a vague query.

Deterministic parser

The rule-based code path in `interpret_command()` that handles clear commands without calling the LLM.

GST Goods and Services Tax — 5% applied to the subtotal.

Jain A dietary restriction excluding all root vegetables, onion, and garlic.

LLM Large Language Model (here: Gemini via Google Generative AI).

Pending state

A queued disambiguation or swap stored in `Session.pending`, resolved by the next user reply.

Session

An in-memory dataclass holding the cart, preference, history, and LLM message list for one customer interaction.

Specials

Items highlighted in `item_availability.json` under `todays_specials`.

Tool call

A structured request emitted by the LLM to invoke one of the nine registered Python functions.

Dakshin Delights Fast Restaurant Assistant — Internal Documentation
© Dakshin Delights Engineering. All rights reserved.